From Sonic Pi to Overtone: Creative Musical Experiences with Domain-Specific and Functional Languages

Samuel Aaron
University of Cambridge
Computer Laboratory
sam.aaron@cl.cam.ac.uk

Alan F. Blackwell
University of Cambridge
Computer Laboratory
alan.blackwell@cl.cam.ac.uk

Abstract

Domain Specific and Functional languages provide an excellent linguistic context for exploring new forms of music notation – not just for formalising compositions but also for live interaction workflows. This experience report describes two novel live coding systems that employ code execution to modify live sounds and music. The first of these systems, Sonic Pi, aims at teaching core computing notions to school students using live-coded music as a means of stimulating and maintaining student engagement. We describe how an emphasis on a functional style improves the ease in which core computer science concepts can be communicated to students. Secondly we describe Overtone, a functional language and live coding environment aimed towards professional electronic musicians. We describe how Overtone's abstractions and architecture strongly benefit from a functional-oriented implementation. Both Sonic Pi and Overtone are freely available open-source platforms.

Categories and Subject Descriptors D.2.6 [*Programming Environments*]: Interactive environments

Keywords Live Coding; Computational Thinking; Pedagogy; Sound Synthesis; Raspberry Pi

1. Introduction

For most children today, their first experience of digital technology is via media devices – being photographed, Skype with their grandparents, or nursery rhymes from YouTube. The same is true of most adults – digital media sometimes seem like little more than a new set of formats for recording, retrieval and transmission. We are challenging this assumption, by giving people creative experiences of computation itself, exposing the functional abstractions that form the underlying basis of aesthetic musical experiences. Mainstream recorded media, even if produced with sampling and remixing tools, is still stuck in the era of the artistic "work" – a concrete thing that is captured and preserved, and represents a static set of sounds and images rather than a structured experience.

Our goal is to provide children, and adults, with new kinds of creative experience by exposing the computation that generates digital music. Music is an excellent choice, because the principles of music cognition and music perception are essentially abstract – when we listen to music, we also hear mathematical patterns. Different kinds of music notation have evolved to express those patterns in different ways. But declarative and functional languages are truly excellent ways to express pattern in digital media. If we can express pattern elegantly, then we can understand it, analyse it and even create it, making our own music. In fact, since children are naturally creative media producers, we suggest that making patterns with functional language is a natural way to teach computational thinking. Seymour Papert's Logo language [13], was based on a similar insight, and has proven an effective educational tool for experimenting with images over many years. However, by adopting more recent domain-specific language techniques, we have been developing new languages that create sophisticated musical abstractions – sufficiently sophisticated to appeal both to children and adult audiences.

The remainder of this paper is divided into two parts. The first introduces Sonic Pi, a simple domain-specific language and IDE for the Raspberry Pi computer, that has been used in classroom experiments with children aged 11-12. It is the first programming language that they have encountered, and their first experiences of programming take the form of music compositions expressed in this language. The second part describes Overtone[1], a more powerful functional language implemented in Clojure[2]. The first author is a leading exponent of live coding – coding generative music systems live in front of an audience. Using Overtone within the heavily customised Emacs variant Emacs Live[3], he creates electro-acoustic compositions and electronic dance music for enthusiastic (adult) audiences.

1.1 Related Research

Programming languages for the specification and performance of music are a very long-standing research topic in computing. One of the earliest pioneers of music programming was Max Matthews, whose programme of research at Bell Labs started in the late 1950s. A long series of experimental music synthesis languages derived from Matthews' work [10] is generically known as "MUSIC-N." The design philosophy evolved through the MUSIC-N series can still be seen in open source systems such as Csound, which uses a C-based syntax to specify combinations of synthesis functions [3]. Paul Hudak has also developed functional approaches to computer music languages in Haskore [8] and Euterpea [7].

A global centre of digital music research is the French computer music research centre IRCAM. Miller Puckette, working at IRCAM, created a dataflow or "patcher"-style digital synthesis en-

FARM '13, September 28, 2013, Boston, MA, USA.

ACM 978-1-4503-2386-4/13/09. . . $15.00.
http://dx.doi.org/10.1145/2505341.2505346

[1] http://overtone.github.io

[2] http://clojure.org

[3] http://overtone.github.io/emacs-live

vironment [14] that has evolved into PureData (Pd)[4] developed by Puckette, and Max/MSP[5] (named in honour of Max Matthews), a visual programming language for music that is now developed and maintained on a commercial basis by Cycling '74.

SuperCollider[6] is a popular tool for real time audio synthesis and algorithmic composition [18] that also offers an execution model based on composition of "unit generators". The SuperCollider architecture separates the sound synthesis server process from a language client that configures the network of unit generators and dispatches control events to the server. The SuperCollider programming language sclang uses a C-like syntax with some object-oriented and functional features. However, many live coders have created alternative interactive languages, such as Thor Magnusson's Ixi Lang, described as a SuperCollider "parasite" [9]. Although we were not previously aware of Thielemann's work on live coding in Haskell [17], we are grateful to one of the referees of this paper for drawing it to our attention, and look forward to discussing it at this workshop.

An introduction to the "first generation" of live coding performance can be found in Collins et al [6]). Many prominent live coders have created their own novel languages, many of which have interesting implementation and functional features, such as Sorensen's Scheme-based language Impromptu [4], more recently extended as Extempore.

2. Sonic Pi Background

2.1 Raspberry Pi

The Raspberry Pi is a very low cost ($25) computer designed primarily to encourage technical experimentation by young people. It is distributed as a bare circuit board, with credit card-sized form factor, that can be used as a full Linux platform with a minimal number of mass-market peripherals (USB mobile phone charger as the power supply, HDMI television as the monitor, an SD card as the main file store, and any USB keyboard and mouse). Since its introduction in 2012, the non-profit Raspberry Pi foundation has sold more than a million Raspberry Pi boards via mainstream electronics distributors.

The main objective of the Raspberry Pi Foundation is educational: to develop genuine technical competence by allowing people to experiment freely with their computers at the hardware and operating system level, rather than being restricted to consumption of digital media via competing locked "ecosystems" of bundled hardware, OS, cloud services and online store. The original goals of the Foundation were to encourage and support those who were interested in such experimentation, through a combination of low cost, easy availability of accessories, and online community support. More recently, the enthusiastic uptake of the Raspberry Pi has attracted the attention of educational campaigners, who see the device as offering an appealing platform for technology education in schools. This is not primarily a cost decision (although the headline cost of the circuit board might appeal to schools, the peripherals are inconvenient for use in classrooms by comparison to bundled educational products). Instead, it is the philosophy of Raspberry Pi as an open media platform that offers schools the opportunity to escape consumption-oriented training in favour of technical and scientific engagement.

2.2 Computational Thinking and the CS Curriculum

The interest in deploying Raspberry Pi in schools comes at a time when national and international groups are campaigning for more sophisticated teaching of computer science. Over the past 20 years, in the UK and elsewhere, schools have gradually reduced the "traditional" teaching of basic programming skills, in favour of understanding computer applications in an educational context (mainly using media authoring tools and interactive teaching materials). In the UK, the Computing at Schools (CAS) campaign group[7] has achieved broad consensus for a school curriculum incorporating the fundamentals of computer science, as they would be recognised in any introductory CS textbook at undergraduate level. Meanwhile, a US-led campaign has advocated recognition of these topics as relevant to all contemporary science – "Computational Thinking" (CT) is advocated as a generic set of skills, analogous to mathematics or literacy.

A consequence of both these movements is that they offer an opportunity to re-think traditional early programming education. The CAS campaign recognises that functional languages have value for teaching CS concepts at least equal to traditional imperative teaching languages. And the CT campaign suggests that a programming language may not even be necessary. As in Tim Bell's CS Unplugged teaching material[8], many fundamental principles of computation can be explored without using a programming language at all. It is in this context that we were invited by the Raspberry Pi Foundation to develop a novel programming language, for use in schools, that would emphasise teaching of fundamental principles of computer science through experiences of creative exploration.

An existing interest in music programming systems led us to propose that music would be an appropriate application area for early programming education. Music continues to be a major component of mainstream popular culture. And music genres popular with younger audiences in the UK (Electronic Dance Music, Dubstep, Grime and others) tend to rely heavily on digital production techniques. In addition to this popular appeal, many qualities of music such as tempo, dynamics, melody, harmony and even timbre are easily expressed in terms of abstract mathematical functions, which means that simple examples of digital music genres can be expressed quite straightforwardly using only basic programming skills. It is this background that provided the starting point for the Sonic Pi project.

3. Sonic Pi Implementation

3.1 Requirement

The objective of the Sonic Pi project was to create a minimal domain specific language (DSL) that would a) provide a musically engaging experience for students; and b) demonstrate a reasonably wide range of basic computational concepts. The target audience was a class of 12-year-olds who had not previously had any programming or other technical computing experience. In the course of five one-hour lessons, we wished to progress from 'this is a computer' to successful completion of an operational program that generated a satisfying piece of music. All syntactic and semantic elements of the language were designed with these primary goals in mind. The resulting DSL has some functional characteristics, but also some imperative features, where those met specific educational objectives. The following sections design the resulting language model and implementation, and also the simple IDE that was created for students to work with it.

3.2 Constraints

Initially, the Sonic Pi project had an extremely tight schedule which only permitted three weeks of development time for version one of the system. The first idea was to port aspects of Overtone (intro-

[4] http://puredata.info/

[5] http://cycling74.com/products/max/

[6] http://supercollider.github.io/

[7] http://www.computingatschool.org.uk/

[8] http://csunplugged.org/

duced in section 4) to run on the Raspberry Pi. Unfortunately, after this had been achieved it was discovered that the performance was several orders of magnitude too slow. This may have been due to a number of issues such as the current JVM's lack of support for floating point operations under ARM6, incomplete JVM garbage collector (GC) support (Clojure requires efficient ephemeral GC collections) in addition to the reduced power of the Raspberry Pi itself. It was therefore necessary to switch to an alternative approach which did not suffer these issues on the Raspberry Pi platform. Ruby was chosen due to the first author being extremely familiar with it. Given the time constraints, choosing a familiar language was clearly more important than spending the appropriate amount of time to research and select an ideal candidate. The Sonic Pi project has now been extended beyond its initial three month period. We are now in a position to consider a more principled approach to language design for future versions.

In addition meeting the tight schedule as explained above, there were also political constraints to consider – it was important to use a language which was used in industry such as Python. In the UK, there is currently a lot of momentum specifically behind Python within teaching contexts. Given the relative semantic and syntactic similarity between Ruby and Python relative to many other languages and styles, it was possible to defend the decision to use Ruby with educators and education stakeholders. However, given that decision it was also important to ensure that programs were written in a somewhat idiomatic style. Idiomatic Ruby is written using a hybrid of functional style (i.e. `map`, `reduce`, `filter` etc.) over collections of stateful Ruby objects. De-emphasising the stateful aspect of Ruby therefore did not feel too much like a departure from this and provided enough linguistic power to represent musical compositions.

3.3 Sonic Pi Language Model

The Sonic Pi language is implemented as a simple embedded Ruby DSL. The principle implementation technique to achieve this is to define and use an object with specific utility methods which both represent the required linguistic functionality (additional to the functionality Ruby already provides) and also setup and teardown methods for external dependencies. In the current Sonic Pi implementation, this context object is a Ruby class named `SpiderLang` which contains a method named `spider_eval` which takes a string as an argument and evaluates it within the context of a given `SpiderLang` instance. This allows the input string to reference the extra linguistic functionality defined within the `SpiderLang` context. With this approach, the main entry point for the language is the following Ruby statement:

```
SpiderLang.new.spider_eval(sonic_pi_code)
```

As Sonic Pi is a music language, it requires a system for generating and manipulating sounds. This system is implemented in terms of the SuperCollider synthesis server which provides an ability to define arbitrary synthesisers and trigger and manipulate them in real time. In its initialisation method (which is triggered by sending the `new` message), the `SpiderLang` class connects to and cleans out the associated SuperCollider server. This ensures that Sonic Pi is ready for new sounds to be triggered and modified by the user.

The easiest way to trigger a new sound with Sonic Pi is to call the `play` method:

```
play 60
```

This will play MIDI note 60 on the current synthesiser[9]. In order to play a simple melody where notes are separated through time, it is necessary to interleave the calls to `play` with calls to `sleep`:

```
play 60
sleep 0.5
play 72
sleep 0.25
play 84
```

This then provides the basis for teaching programming languages as an imperative sequence of operations with a thread of execution – a core requirement of the new UK computer science curriculum. Clearly from a pure functional perspective, this monadic like structure is not necessarily very elegant. However, from our initial experiences in classrooms, it is the time to creating meaningful side-effects which is a major factor in obtaining and holding the attention of the school children. Additionally, a benefit from embedding our language as a Ruby DSL is that we can also take advantage of Ruby's flexible and forgiving parser. This means we can omit parenthesis around method arguments and semi-colons between statements relieving us from a requirement to type (and more importantly not having to explain to keen music-making pupils) the following syntactically noisier but more complete version:

```
play(60);
sleep(0.5);
play(72);
sleep(0.25);
play(84);
```

It is important to point out that each of these core concepts, such as programs as a sequence of statements are introduced to the pupils through a series of acted scenarios. For example, in the case of sequential statements, the pupils are invited to form a row with each pupil holding an instruction card. The first pupil in the line is also given a green control card and is invited to act out the instruction on their card. Once the instruction is enacted (usually involving pretending to sleep for a number of seconds or making an amusing noise), the control card is passed on to the next pupil in the line like a baton in a relay race.

In addition to sequential operations, the Sonic Pi language creates opportunities to expose the children to a number of other core computer science concepts. For example, we can introduce them to lists as basic data structures. The motivation here is that they can represent the notes the want to play as a list. For example, we can re-implement the previous three-note melody with the following:

```
play_pattern [60, 72, 84]
```

This will sleep for a default amount of time between each note (specified in Beats Per Minute, BPM). This then provides us with an opportunity to introduce algorithms as mini-programs which provide useful operations on our data structures. For example, we can shuffle/randomise and reverse our notes:

```
play_pattern [60, 72, 84].shuffle
play_pattern [60, 72, 84].reverse
```

Another concept that Sonic Pi is able to succinctly represent is repetition of actions through iteration. Instead of having to teach local state, counters, conditionals in order to use a traditional for loop we can rely on Ruby's `times` method with an implicit code block:

[9] The current synthesiser is typically set to a pleasing bell sound implemented with additive synthesis techniques

```
10.times do
  play 60
  sleep 0.25
end
```

From an implementation perspective the contents of `do` and `end` are turned into a lexical closure and passed to the `times` method as an implicit parameter. This so-called block can then be called using `yield` within the implementation of `times` and in this case it is called repeatedly according to the cardinality of the number responding to the `times` message.

From a teaching perspective, all of this complexity is irrelevant as it is concealed from the end user. We found the syntax itself facilitated student understanding. For example, , `10.times` reads like plain English and the `do` and `end` tokens can be explained as bookends surrounding the code to be like to repeated. In our initial trials, we observed this form of iteration to have been successfully digested by the pupils trialling Sonic Pi.

This `do` and `end` construct can also be transferred to other concepts. For example, we can represent basic concurrency via the `in_thread` method:

```
in_thread do
  play_pattern [38, 45, 42, 41, 40]
end

in_thread do
  play_pattern [50, 57, 54, 53, 52]
end
```

This might initially appear to be an overly sophisticated concept to introduce. However, it was included within our scheme of work after requests from pupils asking how they can play their melody at the same time as their bass line. Given that the really difficult aspect of threads is coordinating them and the fact that we do not introduce state, there are no nasty pitfalls other than grasping the concept of more than one flow of control executing concurrently. Again, as with the concept of sequential statements, this is knowledge we scaffold by acting this scenario out with cards before they move to the programming language representation.

It is also possible to demonstrate conditionals and randomisation through the production of stochastic musical structures:

```
if rand < 0.5
  play 60
  sleep 0.25
else
  play 72
  sleep 0.5
end
```

Finally, Sonic Pi offers a number of music-specific features such as pads which allow their timbre to be manipulated through mouse gestures[10]. The current synthesiser can be changed to a number of alternative designs each offering different control options such as modifiable attack and decay envelope settings and other parameters which affect the resulting timbre:

```
play 60, "attack", 2, "release", 0.1
```

3.4 Sonic Pi IDE and Synthesis Model

The Sonic Pi language provides a means of formally representing musical intentions to the system. However, this language needs to be entered into the system by the pupils themselves. This is achieved through the bespoke Sonic Pi IDE which was specifically designed to reduce the visual and conceptual complexity typically found within programming editors and environments. The IDE therefore consists of only five components:

- Control buttons
- Workspace tabs
- Editor pane
- Information pane
- Error pane

The control buttons provide the means to start and stop the execution of the program. These exist in the form of familiar triangle start and square stop button similar to those typically found on modern music players. The workspace tabs provide a number of independent workspaces within which to work. The main motivation here is to provide a new workspace for each separate lesson whilst also allowing the pupils to access the workspaces used in the previous lessons to recall prior learning. The editor pane is where they write their programs and provides basic syntax highlighting, the information pane provides real-time feedback during the execution of a program to indicate to the user what it is doing. Finally, the error pane provides a means for the Sonic Pi language to report syntax and runtime errors to the user.

It is important to briefly discuss what is not part of the editor's functionality. For example, there is no file save system as the IDE automatically saves the contents of the current workspace. There is no project system, no menus, no macro system, no refactoring wizard and no irc client. This makes the IDE easy to understand and operate both by pupils and also non-expert IT teachers.

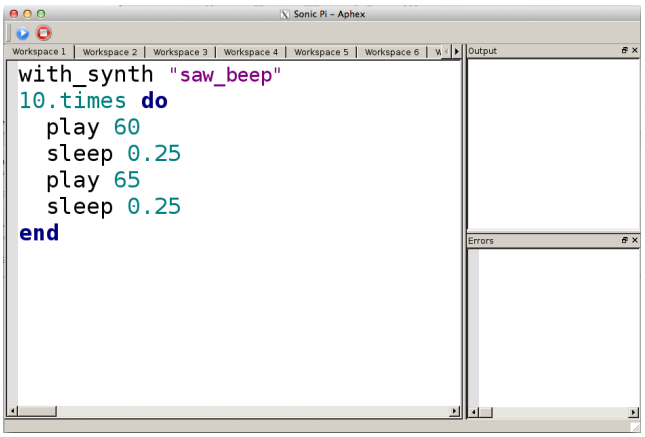

Figure 1. The Sonic Pi IDE

3.4.1 Synthesis Model

In order to generate sound, Sonic Pi uses the SuperCollider synthesis engine. This engine is a real-time low-latency live modifiable server which is communicated to via Open Sound Control (OSC) messages typically transmitted via UDP. The benefit of using SuperCollider is that it provides a very powerful and flexible synthesis foundation for us to work with in the future if we were to focus explicitly on Sonic Pi as an instrument in addition to being a tool for teaching basic programming concepts.

From a systems perspective Sonic Pi can therefore be perceived as an OSC client which speaks a subset of the full SuperCollider OSC API. The implemented subset is sufficient to trigger, control synthesisers and also to clear out the server between program runs.

[10] The statement `play_pad "saws", 38` will play a saw-tooth pad with the frequency set to MIDI note 80

The synthesisers themselves are data structures representing specific designs and stored in a binary format with a separate file for each synthesiser. Sonic Pi currently has no capability for designing these synthesisers and therefore relies on the existence of an external program such as the original SuperCollider language or Overtone to create a library.

SuperCollider server (discussed in further detail in section 5) is an extremely stateful entity and to work with it successfully requires much of that state to be mirrored in the client environment. Sonic Pi therefore represents this state, but keeps it hidden from the user.

4. Overtone Background

Overtone is a functional language that, like Sonic Pi, is designed to be used for musical composition. However, whereas Sonic Pi is primarily an educational language, intended for use in classrooms with a specific teaching purpose, Overtone has been designed for use in arts contexts by professional artist/programmers. The original design objective was to support Live Coding [6], a performance art form in which a programmer creates code for a live audience. Live Coding is largely associated with musical performance, although many live coders also create animated images or processed video mixes (sometimes performing in a group with a live coding musician, as in the work of slub[11], or audiovisual duo klipp av [5]).

4.1 Key Abstractions in Overtone

Overtone [2] is a much broader realisation of many of the ideas found in the Sonic Pi system in that it is essentially an OSC client for the SuperCollider server. However, it implements the full OSC API and provides many abstractions for representing and manipulating the server at a much higher level. These abstractions have been specifically designed to facilitate the interactivity required for live coding and the coordination of external events such as those generated by a performer.

Overtone is implemented in Clojure, a modern Lisp primarily targeted for the JVM. Clojure is a functional programming language emphasising (although not requiring) pure functions, immutable data structures, higher order functions. It is strictly but not statically typed relying on the JVM to provide the underlying types themselves. We argue that it is the strong functional aspects of Clojure which have allowed the implementation to be extremely maintainable despite its complexity and also provides extremely stable foundations for continuing innovation for both music representation and interaction.

This section will introduce the basic abstractions that Overtone provides with a specific emphasis on their functional aspects such as their representation in terms of immutable data structures and functions.

4.1.1 Records

One of the goals of an environment with a high level of liveness [16] is the ability to modify any aspect of the system at run time. Overtone provides this via Clojure's code hot-swapping functionality combined with dynamic typing. Statically typed environments such as Haskell can exhibit some dynamically typed behaviour, for example, via Data.Dynamic and the Data.Typable modules. However, new types cannot be dynamically created nor old types have their definition changed. Additionally, it is not possible to redefine functions on-the-fly (although the GHC REPL provides similar behaviour, each function redefinition actually creates a separate new version of the function and any references in other function bodies still point to the original function definition.) This is not the case with Clojure which allows for new types to be created at runtime and for any aspect of functions, including their signature to be modified at run time. This power to modify the system introduces a new class of problems which represent new research opportunities for live languages.

Overtone makes extensive use of Clojure's record construct which provide a means for structuring key concepts in an extensible manner, yet providing underlying types which can be introspected at run time. These types are typically used as a basis for providing polymorphic behaviour via Clojure's Protocol system. Protocol's place the polymorphism in the function using the type of the first argument as the dispatch strategy. This allows us to implement a suite of general functions such as `kill` which provide a variety of different implementations for different records and types. This is much of the benefit of an Object Oriented approach to dispatch in a functional style.

4.1.2 Unit Generators

Overtone's most basic music synthesis abstraction is the `SCUGen` record which is an abbreviation for SuperCollider unit generator. This concept dates back to the early 60s when Max Mathews described a notation for representing the components of synthesis design [10]. SuperCollider currently has over 500 unit generators representing the basic building blocks of synthesisers such as oscillators, filters and generators. Each of these unit generators is implemented in C and is separately compiled for all platforms. In Overtone a `SCUGen` starts its life as metadata which describes its controllable arguments (if any) and documentation. For example, the metadata for the sine oscillator unit generator is represented using the following EDN[12] syntax:

```
{:name "SinOsc",
   :args [{:name "freq",
           :default 440.0
           :doc "Frequency in Hertz"}

          {:name "phase"
           :default 0.0
           :doc "Phase offset or modulator in
                 radians"}]

   :summary "Sine table lookup oscillator"

   :doc "Outputs a sine wave with values
         oscillating between -1 and 1 similar
         to osc except that the table has
         already been fixed as a sine table of
         8192 entries.

         Sine waves are often used for creating
         sub-basses or are mixed with other
         waveforms to add extra body or bottom
         end to a sound. They contain no
         harmonics and consist entirely of the
         fundamental frequency. This means that
         they're not suitable for subtractive
         synthesis i.e. passing through filters
         such as a hpf or lpf. However, they are
         useful for additive synthesis i.e.
```

[11] http://slub.org/

[12] EDN (http://github.com/edn-format/edn) is a subset of the Clojure syntax. It includes syntax for basic values such as numbers `1400`, strings `"foo bar"`, keywords `:foo`, and nestable datastructures such as lists `(:a :b :c :d)`, vectors `[1 2 3 4]`, associative maps `{:foo 1, :bar [2 3]}` and sets `#{:a :b "cheese"}`. EDN ignores commas, treating them as whitespace.

```
    adding multiple sine waves together at
    different frequencies, amplitudes and
    phase to create new timbres." }
```

This metadata is then converted into a number of helper functions which are then composed to form a function which when called will perform validation, argument manipulation and other actions before finally returning a `SCUGen` record.

4.2 Synthesisers

Unit generators in isolation are not useful. They need to be composed into a tree (see figure 2 for an example) before their functionality becomes valuable. For example, the `sin-osc` oscillator needs to be fed into the `out` ugen for it to be made audible. This is achieved by passing one as the argument of the other:

```
(out 0 (sin-osc))
```

The semantics here are the typical eager evaluation of Clojure. The `sin-osc` function is called with no arguments, therefore the defaults will be employed and an appropriate `SCUGen` record is created. This is then passed in turn as the second argument to the `out` ugen, the first argument being which bus to output to (bus 0 represents the left speaker). This tree of `SCUGens` is a simple immutable data structure which then can be converted into a synth design and therefore given a name:

```
(defsynth foo [] (out 0 (sin-osc)))
```

The `defsynth` macro then converts the `SCUGen` tree into a flatter data structure closely resembling the final binary format SuperCollider server expects:

```
{:name "user/foo",
 :params (),
 :n-ugens 2,
 :ugens
 ({:name "out",
   :rate :ar,
   :special 0,
   :inputs {:signals [{:name "sin-osc", :id 0}],
            :bus 0.0},
   :n-outputs 0,
   :outputs (),
   :id 1}
  {:name "sin-osc",
   :rate :ar,
   :special 0,
   :inputs {:phase 0.0, :freq 440.0},
   :n-outputs 1,
   :outputs ({:rate 2}),
   :id 0})}
```

This datastructure is then compiled to binary and sent to SuperCollider server via UDP. Once this has completed, `defsynth` creates a function in the current namespace with the specified name (`foo` in this case) which can then be used to trigger new instances of the synthesiser:

```
(foo) ;=> a simple sine tone is created
```

To stop this sound, the function `(stop)` can be used which stops all running synthesisers.

4.2.1 Controlling Synthesisers

The trigger function `foo` returns a `SynthNode` record which can be captured:

```
(def running-node (foo))
```

This can then be stopped individually with the `kill` function:

```
(kill running-node)
```

The `SynthNode` record contains a stateful field representing its current state. It is initially `loading` until SuperCollider server sends an asynchronous message to indicate that the synthesiser is executing after which the field is updated to `running`. Once killed, the state moves to `destroyed`. This run-time information is extremely useful in a live coding context where maintaining a coherent overview of the current state of the system is a key requirement.

Once a synthesiser is executing, it is possible to control it. It may be paused and re-started with `node-pause` and `node-start`. However, the real power comes when you add control parameters to the synthesiser. For example, consider the following design:

```
(defsynth beep
  "A simple audible sine wave tone"
  [freq 440 amp 1]
  (let [snd (sin-osc freq)]
    (out 0 (pan2 (* snd amp)))))
```

This introduces two new concepts. Firstly, we have included a docstring which eventually becomes part of the docstring of the generated trigger function. Secondly, we define some control parameters: `[freq 440 amp 1]`. The symbols `freq` and `amp` in the vector represent the control parameters and are valid for use within the body of the `defsynth` macro similar to the bindings of a standard let statement. Figure 2 illustrates the final form of the tree representing this design.

Now that the synthesiser has control parameters, these can be used when triggering it to start it with different settings:

```
(beep :freq 220 :amp 0.5)
(beep :freq 880 :amp 0.25)
```

These control parameter may also be used to control a running synthesiser via the `ctl` (an abbreviation for control) function:

```
(def b (beep))
(ctl b :freq 220)
(ctl b :freq 880)
(ctl b :amp 0)
(kill b)
```

We now have something that is starting to resemble the Sonic Pi software[13].

4.2.2 Synthesiser Design with HOFs

Synthesiser designs may employ higher order functions (HOFs) to allow complex structures to be described in a more concise and simple form. For example, consider the following synthesiser design which was originally designed in SuperCollider Language (see section 5.1) by Fredrik Olofsson[14] and was subsequently translated to Overtone by the first author:

```
(defsynth redfrik-3296
    []
    (let [part
          (fn [i]
            (let [j (sin-osc (+ i 1/99))]
```

[13] The fundamental difference here is that given a powerful text editor such as Emacs, it is possible to evaluate these forms in an arbitrary order at arbitrary times. This interaction starts to feel more like playing an instrument than coding.

[14] https://twitter.com/redFrik/status/329680702205468674

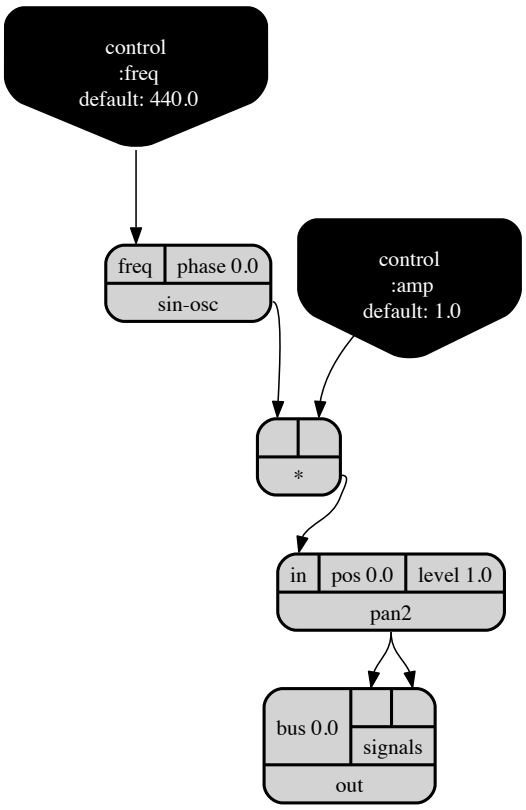

Figure 2. The beep synth design

```
    (leak-dc (pan2 (* j
                      (sin-osc
                       (+ i 1/9)))
                   (* j (sin-osc
                         (* (+ 39
                               (* 39
                                  (ceil j)))
                            (+ i 1 0))
                         (sin-osc
                          (+ 2 0)))))))]
(out 0 (/ (mix (map part (range 9))) 3))))
```

The syntactic structure of this synthesiser may not appear much more complex than the earlier beep synthesiser. However, it is employing HOFs to generate a more complex design when it is compiled to its final form. Contrast the complexity of the unit generator structure presented in Figure 3 with that in Figure 2.

4.2.3 Composed Generators

Another approach to managing the complexity of synthesiser designs is via the `CGen` abstraction. A `CGen` is a tree of `SCUGens` presented as if it were a unit generator itself. This abstraction is not too dissimilar from the abstract of a function over a given computation. Similar to function composition, `CGens` may also be composed, allowing for potentially very complex designs be represented by a series of more suitable high-level constructs.

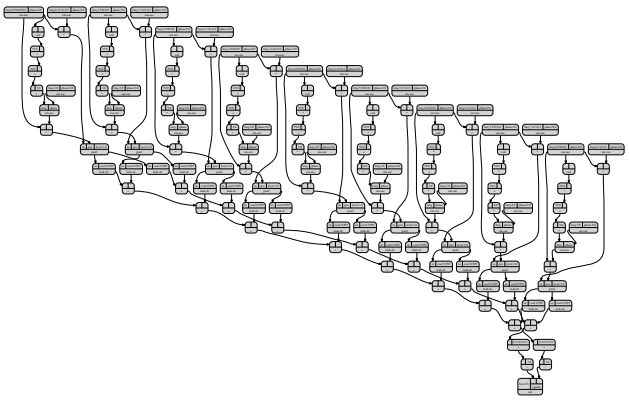

Figure 3. The redfrik-3296 synth

For example, consider this `CGen` representing the partials of a bell:

```
(defcgen bell-partials
  "Bell partial generator"
  [freq {:default 440
         :doc "The fundamental frequency for the
               partials"}
   dur {:default 1.0
         :doc "Duration multiplier. Length of
               longest partial will be dur
               seconds"}
   partials {:default [0.5 1 2 4]
             :doc "sequence of frequencies which
                   are multiples of freq"}]
  "Generates a series of progressively shorter
   and quieter enveloped sine waves for each of
   the partials specified. The length of the
   envelope is proportional to dur and the
   fundamental frequency is specified with freq."
  (:ar
   (apply +
          (map
           (fn [partial proportion]
             (let [env      (env-gen
                             (perc 0.01
                                   (* dur
                                      proportion)))
                   vol      (/ proportion 2)
                   overtone (* partial freq)]
               (* env vol (sin-osc overtone))))
           partials
           (iterate #(/ % 2) 1.0)))))
```

This creates a function named `bell-partials` which can be used exactly like a standard unit generator function. In this case the function has three parameters, `freq`, `dur` and `partials`. The implementation uses these parameters to construct a tree of `SCUGens`.

We can use it to create a pretty bell sound by supplying mostly harmonious partials[15] (specified as a vector of numbers in the first line of the `let` binding):

```
(defsynth pretty-bell [freq 220 dur 1.0 vol 1.0]
   (let [ps  [0.5 1 3 4.2 5.4 6.8]
         snd (* vol (bell-partials
                     freq
```

[15] A harmonious partial is an integer multiple of the fundamental frequency.

```
                    dur
                    ps))]
      (detect-silence snd :action FREE)
      (out 0 snd)))
```

We may also a create a dull bell sound by supplying inharmonious partials

```
(defsynth dull-bell [freq 220 dur 1.0 vol 1.0]
  (let [ps [0.56 0.92 1.19 1.71 2 2.74 3 3.76 4.07]
        snd      (* vol (bell-partials
                         freq
                         dur
                         ps))]
    (detect-silence snd :action FREE)
    (out 0 snd)))
```

4.2.4 Notes, Scales & Chords

Once we have the ability to generate and manipulate synthesisers, we are free to use the powerful functional capabilities of Clojure to design a suite of higher level abstractions more suited to the representation of melody and composition. For example, one of the built-in synthesisers is the `sampled-piano`. This allows the piano to be played as a series of function calls by applying the function to a MIDI note:

```
(sampled-piano 60)
(sampled-piano 62)
(sampled-piano 64)
(sampled-piano 65)
```

We can then conceive of more suitable abstractions. For example, Overtone has a namespace containing functions related to the structure and composition of Western melodies. One of these functions is `note` which takes a human readable keyword such as `:C4` which avoids the use of magic numbers:

```
(sampled-piano (note :C4))
(sampled-piano (note :D4))
(sampled-piano (note :E4))
(sampled-piano (note :F4))
```

We can easily represent scales and chords and apply computation over them. For example, the first four notes in the scale C4 major, followed by the chord C4 major7:

```
(doseq [n (take 4 (scale :C4 :major))]
  (Thread/sleep 200)
  (sampled-piano n))

(doseq [n (chord :C4 :major7)]
  (sampled-piano n))
```

4.2.5 Functional Composition

Another application of functional programming languages for composition is the use of HOFs for representing structure. This idea has been explored by Chris Ford in Leipzig[16], an Overtone library. Leipzig provides a high-level interface for composing melodies which makes extensive use of this approach. For example, consider the following which uses HOFs such as `phrase` to represent a series of notes and corresponding times, `then` to thread them together and functions such as `where` which in this case apply some filtering:

```
(def melody
  "A simple melody built from durations and
   pitches."
              ; Row, row, row your boat,
  (->> (phrase [3/3 3/3 2/3 1/3 3/3]
               [  0   0   0   1   2])
    (then
              ; Gently down the stream,
       (phrase [2/3 1/3 2/3 1/3 6/3]
               [  2   1   2   3   4]))
    (then
              ; Merrily, merrily,
              ; merrily, merrily,
       (phrase (repeat 12 1/3)
               (mapcat (partial repeat 3)
                       [7 4 2 0])))
    (then
              ; Life is but a dream!
       (phrase [2/3 1/3 2/3 1/3 6/3]
               [  4   3   2   1   0]))
    (where :part (is :leader))))
```

4.2.6 Concurrent Event System

Another aspect of musical interaction where functional approaches provide a strong advantage is that of event handling and manipulation. Overtone incorporates an event system where handlers are basic functions which take one argument – an immutable associative map of event information. This map may be arbitrarily large, and an arbitrary number of matching handler functions may be executed each taking an arbitrary amount of time to complete. Despite all this, the immutable data structures prevent event handlers from interfering with each other, only references to the data structures are passed to each handler which is both extremely cheap and fast and finally the default strategy for the event system is to run each handler on an arbitrary thread on a thread pool allowing for concurrent handling of events. All of this would be much harder and much more error-prone in a traditional mutable language.

For example, consider the following multi-threaded implementation of a piano handling events from an externally connected MIDI keyboard:

```
(on-event [:midi :note-on]
          (fn [msg]
            (sampled-piano (:note msg)))
          ::simple-piano)
```

Overtone supports a variety of semantics for handling events. As described above, `on-event` results in the handler being executed within a thread-pool for matching events. There is also `oneshot-event` which is guaranteed to execute the handler only once for the first matching event, `on-sync-event` which will execute the handler synchronously on the same thread as the event itself and `on-latest-event` which will execute asynchronously on a thread-pool but not necessarily for every event – if the time between incoming events is less than the execution time of the handler, intermediate events will be dropped (i.e. not handled) in order to stop backlogs occurring and reduce latency. It is also possible to specify the concurrency semantics from the thread creating the event. Calling `event` will result in the default behaviour, whilst calling `sync-event` will force all matching handlers to execute on the current thread, regardless of whether they were declared as asynchronous or not.

4.2.7 Event Persistence

Representing events as non-blocking immutable data structures also has the benefit of being easily stored in memory or on disk to allow specific event streams to be persisted and subsequently

[16] https://github.com/ctford/leipzig

queried as demonstrated in the Snapshots system [1]. This facilitates additional independent threads of execution querying past events with potentially expensive algorithms without blocking the system from continuing as normal. For example, when a specific event occurs, it may trigger two handler functions, one which instantly triggers a synthesiser and another which runs an analysis on a large amount of previous events to look for specific patterns. As the events are represented as an immutable data structure, this second long-running process may operate over a specific snapshot of events correct at the time the handler was triggered without that data structure ever changing all while new events may continue to enter the system.

4.2.8 Temporal Recursion

Finally, Clojure allows for an elegant implementation of the temporal recursion pattern [15]. This is essentially a recursive form which rather than a tail call to the current function, is scheduled call to the current function to be applied at some future time. This creates a co-routine like structure which can be triggered and manipulated at run time by modifying the function itself.

Overtone's temporal recursion is built upon `apply-at`, a special version of Clojure's `apply` fn which not only applies the specified function to the given arguments but also schedules the application for a future time on a potentially different thread. The following call applies the function `println` to the argument list `["hello world"]` in 1000ms from now which results in the string `"hello world"` printed to the standard out stream :

```
(apply-at (+ (now) 1000) println ["hello world"])
```

The function `apply-at` may be used to implement a recurring schedule by threading time as arguments. In the following example, we define a function which takes two parameters, the current time and a separation time. The function performs its operation – in this case, printing `"hello world"` and then calcuates the next time which is the summation of the current time and the separation time. It then uses `apply-at` to schedule itself for execution at this new time:

```
(defn scheduled-hello-world [curr-t sep-t]
  (println "hello world")
  (let [new-t (+ curr-t sep-t)]
    (apply-at new-t
              scheduled-hello-world
              [new-t sep-t])))
```

An idiomatic approach to accurately scheduling music in Overtone is to combine `apply-by`, a variant of `apply-at` which applies the function slightly ahead of the specified time, with the Overtone `at` macro. The `at` macro will evaluate its body and will wrap any resulting OSC messages to the SuperCollider server in an OSC bundle with the specified time-stamp rather than sending them individually. In accordance to the OSC specification, the SuperCollider server delays the evaluation of the OSC bundle and its containing messages until that time. SuperCollider's implementation is specifically designed to make the time of this execution as accurate as possible which is unlike `apply-at` which is subject to garbage collection (GC) pauses and unpredictable Java thread context switch times. Assuming that the time `apply-by` executes ahead of time is greater than the sum of any GC and context switch times, this gives us direct access to the temporal accuracy SuperCollider server is able to provide.

As an example of temporal recursion in Overtone combined with some of the available functional musical abstractions, here is a simple implementation/composition of a simple Reich phase pattern[17]. A typical Reich phase pattern takes a sequence of notes such as that represented in figure 4 and plays it with two performers concurrently with one performer playing the pattern fractionally faster than the other. Initially, this has the effect of both performers playing appearing to be synchronised. However, due to being played at different rates, they gradually shift out of phase and then back in phase with the faster pattern one note ahead.

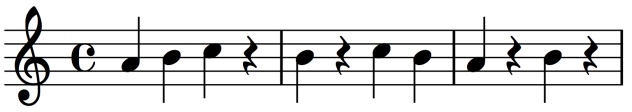

Figure 4. A Reich phase pattern

We can represent the notes in figure 4 with the following vector of keywords which represent the pitch separations between the notes as roman numerals with underscores representing rests[18]:

```
(def reich-degrees [:vi  :vii :i+  :_
                    :vii :_   :i+  :vii
                    :vi  :_   :vii :_])
```

We can now project the degrees to MIDI notes within a specific key and scale using the `degrees->pitches` function:

```
(def pitches (degrees->pitches reich-degrees
                               :diatonic
                               :C4))
```

Next, we implement our temporal recursion. This is a function called `play` which takes three arguments: the time to play a given note, the notes to play and the separation time between notes. The body of the function grabs the first note, checks whether it is a rest and if not, it schedules an instrument to play it at the specified time. Finally, it uses `apply-by` to schedule itself to execute slightly ahead of the new time which is the time plus the separation time. The notes to play are specified as the tail of the notes. The separation time is passed through unmodified:

```
(defn play
  [time notes sep]
  (let [note (first notes)]
    (when note
      (at time (plucked-string note)))
    (let [next-time (+ time sep)]
      (apply-by next-time
                play
                [next-time (rest notes) sep]))))
```

We may now use the `play` function to play our note pattern:

```
(play (now) pitches 200)
```

Finally, we can convert this to a Reich phase by starting two patterns simultaneously with slightly different separation times. We can also convert our finite list of notes into an infinite lazy sequence of notes using Clojure's `cycle` function..

```
(let [t (+ 100 (now))]
  (play t (cycle pitches) 200)
  (play t (cycle pitches) 201))
```

[17] https://en.wikipedia.org/wiki/Piano_Phase

[18] Roman numerals were chosen to reflect the typical notation used by musicians for degree within scales.

5. Supercollider

Underlying both Sonic Pi and Overtone is the SuperCollider synthesis engine [11]. This is a realtime highly modifiable audio synthesis engine. SuperCollider itself ships as two major components the language and the synthesis server. This separation was specifically to support the exploration of new languages other than the default SuperCollider Language such as Ruby and Clojure[12]. These two components are highly decoupled with all communication between them carried out in the Open Sound Control protocol[19]. This section will briefly describe these two components specifically focussing on their relationship with both Sonic Pi and Overtone

5.1 Language

The SuperCollider language is similar to Ruby in its design and heritage. It is a fully object-oriented language derived from Smalltalk with first-class functions and a library of Higher Order Functions (HOFs) for functionally chaining computation through functions such as `map`, `filter`, `reduce` etc. It is possible to write extremely terse and expressive programs in the SuperCollider language. However, it suffers from the same kind of usage issues as Ruby. For example, while it may be possible to effectively write code that communicates intent, the highly mutable and polymorphic nature of the language makes it much harder to communicate the precise operational semantics of the code. Understanding the behaviour of programs rapidly becomes a non-trivial problem often requiring the conceptualisation of complex graphs of dependencies. We therefore believe that both the SuperCollider language and Ruby are perfectly suited for building training environments where a full understanding of the operational semantics is not essential. Additionally, attempting to build multi-threaded programs with these languages is extremely error-prone and difficult due to the inherent mutability of the program and the objects the program operates over.

After experimenting with a number of approaches to building large sophisticated multi-core musical systems, the first author has had far greater success with much less mutable and more functional languages such as Clojure.

5.2 Communicating with Overtone

Unlike Sonic Pi, Overtone does not come with an integrated editing environment. Instead, there are three main methods of communicating with a running Overtone process: the REPL, loading source files and arbitrary form evaluation. These provide different approaches to either insert new functionality or hot-swap with existing code.

5.2.1 REPL

The simplest of these options is the REPL which is bundled with the Clojure build tool Leiningen[20]. This REPL is a simple program which connects to an nREPL (networked REPL) server running within the Clojure process on the JVM. The REPL supports the standard Read Evaluate Print Loop using the network to send the form read in from the prompt to the Clojure server for evaluation. Once the response has been received, it prints the result to the screen. Whilst this approach is trivial to set up, the REPL workflow is only suitable for simple command response cycles with the design and manipulation of more sophisticated algorithms being more cumbersome.

5.2.2 Loading Source Files

For larger programs it can be more convenient to use a standard text editor to write the code and save it to a file, typically with a `.clj` extension. This file may then be loaded into a running Clojure process by using the load command within a connected REPL. This then instructs Clojure to load and compile the entire file at once.

5.2.3 Arbitrary form evaluation

The final option is a combination of the first two. Code is written into an editor and arbitrary forms may then be selected and evaluated. This approach requires a compatible editor such as Eclipse, Vim and Emacs. The text file may be treated as a temporary scratch environment for experimentation, a key file implementing a core subsystem or more typically somewhere in between. Code can therefore start in a messy experimental state and naturally transition into something more structured and stable.

5.3 Emacs Live

An excellent environment for experimenting with live coding workflows is the venerable Emacs editor. Its highly modifiable architecture enables users to live-code Emacs using Emacs itself using the arbitrary form evaluation style described above. There are also a large number of external libraries and tools written by a wide community of Emacs developers which provide extremely useful functionality for live coding. For example, there is the `undo-tree` library which replaces the default Emacs undo ring[21] with a tree structure which can be rendered in ascii-art and interactively navigated. This provides functionality similar to version control at a much finer granularity. Other functionality provided by external tools includes support for highlighting the specific forms evaluated, inline documentation which is not generated from static analysis but is dynamically obtained at run-time, instant navigation to the implementation of functions and much more. The first author has packaged many of these tools ensuring compatibility and has released them under the Emacs Live project. Installing Emacs Live is as simple as downloading a directory and using it to replace `~/.emacs.d`. Emacs Live has a vibrant and growing community which includes not only artistic programmers but also professional programmers. For example it is used by a large number of the entertainment division at Nokia for their internal Clojure development.

6. Evaluation

In this section we will provide preliminary evaluations both Sonic Pi and Overtone discussing opportunities for further research where necessary.

6.1 Preliminary findings from Sonic Pi

At the time of writing, Sonic Pi has been used to deliver a completed series of lesson plans for two classes at a pilot school. Both classes were in Year 7 at a state-funded school (11-12 years old, UK national curriculum Key Stage 3), with one composed of relatively high ability students, and the other relatively low ability. These groups had already been separated within their school by ability. The overall plan of work consisted of five one-hour lessons, each supported by a detailed lesson plan of classroom activities leading to defined learning outcomes. The lessons were taught by the first author, together with the regular classroom teacher. The school, and these classes, were selected for the pilot study in response to the classroom teacher's personal enthusiasm for the Raspberry Pi project.

This pilot study has demonstrated the feasibility of the approach, and the usability of the Sonic Pi DSL and development environment for classroom use. Detailed in-class observation and evaluation was not included in this pilot study, mainly for budgetary reasons. However, all students successfully acquired basic

[19] http://opensoundcontrol.org/

[20] http://leiningen.org/

[21] Emacs typically stores a history of modifications in a ring buffer.

competence in programming that is relatively impressive for such a short course. This was demonstrated with a final evaluation lesson in which the students were tasked to write a functioning program using all of the concepts that they had been taught, which they all completed successfully. This competence included not only conventional introductory programming constructs, but also advanced topics such as multi-thread execution that are not normally taught at school level, but are very naturally introduced in the context of musical abstractions. Perhaps more importantly, the students were extremely enthusiastic about this first encounter with the world of programming. They created their own satisfying (if simple) musical compositions, and were inspired by the low cost and openness of the Raspberry Pi platform – a computer that they could see and assemble themselves, rather than hidden behind a touchscreen or web browser.

Scientific validity of a single pilot is relatively low, in part because the creator of the Sonic Pi was present in the class to guide the teacher and students. However this work is now being replicated in two further schools, by teachers not involved in the original pilot, based only on the written and other support materials that have resulted from the pilot. We are receiving increasing numbers of enquiries from other schools wishing to experiment with Sonic Pi, including music education coordination bodies, and are cautiously optimistic that it may become more widely popular.

6.2 Live Coding Performances With Overtone

Overtone has created a considerable impact within the Clojure community and beyond. The first author received a standing ovation after introducing it to the official Clojure conference in 2011 to a crowd of professional programmers. It has since been used as a way of demonstrating live programming techniques to technical audiences at a large number professional conferences and user groups. Overtone was recently used to introduce Clojure and function composition in latest official Clojure workshop video serious published by O'Reilly.

In addition to using Overtone to augment presentations and keynotes, it has seen adoption as a live electronic platform. For example, the band Meta-eX[22], started by the first author, has performed at a wide range of venues from professional conferences to established art venues:

- Kiev, Ukraine – Hotcode, 1st June 2013
- London, UK – Music Tech Fest, 18th May 2013
- London, UK – Algorave, MS Stubnitz, 16th May 2013
- Zurich, Switzerland – GOTO Con Zurich, Zurich Marriott Hotel, April 2013
- London, UK – Emacs Conf, March 2013
- London, UK – TechMesh, Russell Hotel, December 2012
- Tampere, Finland – Tampere Goes Agile, Uusi Tehdas, October 2012
- Prague, Czech Republic – Open Suse Developer Conference, Klub Lavka, October 2012
- Bristol, UK – Arnolfini Art Gallery, July 2012

Meta-eX continues to push the boundaries of what is possible with Overtone and is being used as one of the main drivers for new functionality. New features are therefore developed in response to real musical requirements.

[22] http://meta-ex.com

7. Conclusion

We have described two languages, Sonic Pi and Overtone, that can be used to express musical abstractions and create music on the fly. Sonic Pi is a simple DSL intended for use by children in classrooms, running on a very low cost open hardware platform. Overtone is a sophisticated functional language that can be used for synthesis of professional quality sound in live performances. Both languages make use of the SuperCollider audio synthesis architecture. We have demonstrated that domain-specific and functional language design principles can be used in conjunction with such an architecture to realise educational and artistic goals through music.

Acknowledgments

We would like to thank the Raspberry Pi Foundation for funding the development of Sonic Pi. We would also like to thank the many wonderful Overtone contributors, especially Jeff Rose for breathing life into the project. Finally we would also like to thank our reviewers, Paul Hudak and Alan Mycroft for their helpful comments on earlier drafts.

References

[1] S. Aaron and J. Judge. Snapshots: New Possibilities For Social Digital Music-Making Arising From The Storage Of History. In *Proceedings Of The International Computer Music Conference*, pages 228 – 235, 2012.

[2] S. Aaron, A. F. Blackwell, R. Hoadley, and T. Regan. A principled approach to developing new languages for live coding. In *International Conference on New Interfaces for Musical Expression*, pages 381–386, 2011.

[3] R. Boulanger. *The Csound Book: Perspectives in Software Synthesis, Sound Design, Signal Processing,and Programming*. The MIT Press, 2000. ISBN 0262522616.

[4] A. R. Brown and A. Sorensen. Interacting with Generative Music through Live Coding. *Contemporary Music Review*, 28(1):17–29, Feb. 2009. ISSN 0749-4467. .

[5] N. Collins and F. Olofsson. klipp av: Live Algorithmic Splicing and Audiovisual Event Capture. *Computer Music Journal*, 30(2):8–18, 2006.

[6] N. Collins, A. Mclean, J. Rohrhuber, and A. Ward. Live coding in laptop performance. *Organised Sound*, 8(3):321–329, 2003. .

[7] P. Hudak. *The Haskell School of Music – From Signals to Symphonies*, volume 4. (Version 2.5), 2013.

[8] P. Hudak, T. Makucevich, S. Gadde, and B. Whong. Haskore Music Notation – An Algebra of Music. *Journal of Functional Programming*, 6(3):1–19, 1996.

[9] T. Magnusson. The Ixi Lang: A SuperCollider Parasite for Live Coding. In *International Computer Music Conference*, 2011.

[10] M. V. Mathews. The Digital Computer as a Musical Instrument. *Science (New York, N.Y.)*, 142(3592):553–7, Nov. 1963. ISSN 0036-8075. . URL `http://www.ncbi.nlm.nih.gov/pubmed/17738556`.

[11] J. Mccartney. SuperCollider : a new real time synthesis language. *International Computer Music Conference*, pages 257–258, 1996.

[12] J. McCartney. Rethinking the Computer Music Language: SuperCollider. *Computer Music Journal*, 26(4):61–68, Dec. 2002. ISSN 0148-9267. .

[13] S. Papert. *Mindstorms: Children, Computers and Powerful Ideas*. BasicBooks, 1993. ISBN 9780786723881. URL `http://books.google.co.uk/books?id=HhIEAgUfGHwC`.

[14] M. Puckette. The Patcher. *International Computer Music Conference*, pages 420–429, 1988. URL `http://www.citeulike.org/group/12573/article/6483577`.

[15] A. Sorensen and H. Gardner. Programming With Time Cyber-physical programming with Impromptu. *Proceedings of the ACM*

international conference on Object Oriented Programming Systems Languages and Applications, pages 822–834, 2010. . URL http://doi.acm.org/10.1145/1869459.1869526.

[16] S. L. Tanimoto. VIVA: A visual language for image processing. *J. Vis. Lang. Comput.*, 1(2):127–139, June 1990. ISSN 1045-926X. . URL http://dx.doi.org/10.1016/S1045-926X(05)80012-6.

[17] H. Thielemann. Live music programming in Haskell. *CoRR*, abs/1303.5:1–10, 2013.

[18] S. Wilson, D. Cottle, and N. Collins. *The SuperCollider Book*. The MIT Press, 2011. ISBN 978-0-262-23269-2.